

Tema 3.

Transporte de datos

Facultad de Ciencias Exactas, Naturales y Agrimensura UNIVERSIDAD
NACIONAL DEL NORDESTE

REDES DE DATOS -

Capa de Transporte

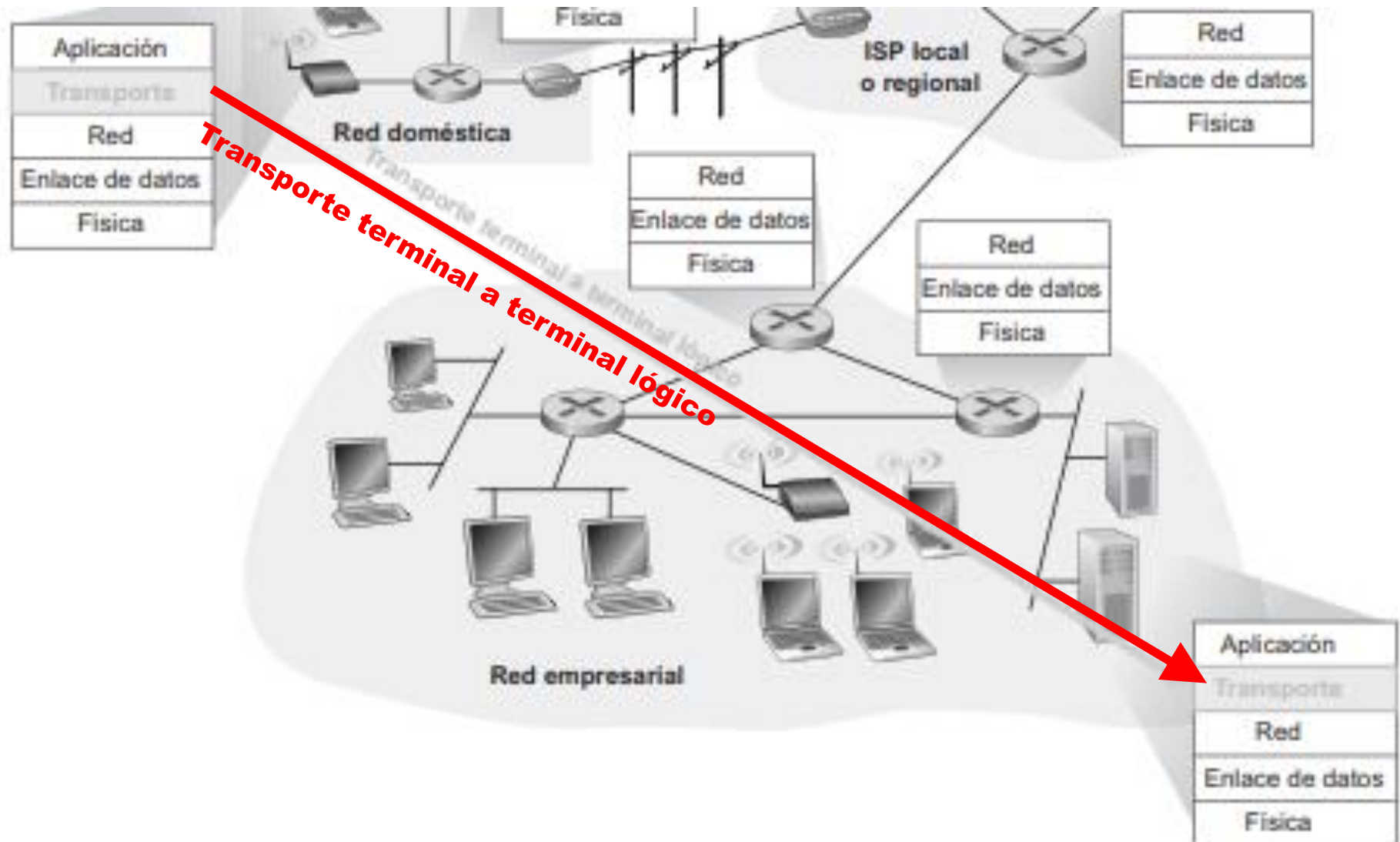
3.1 Interconexión de redes LAN y WAN. El modelo de Internet.

3.2 Protocolo de Transporte UDP

3.3 Protocolo de Transporte TCP

3.4 Mecanismos de sockets y puertos. Paso de mensajes.

Capa Transporte – Interconexión de redes LAN y WAN. El modelo de Internet.

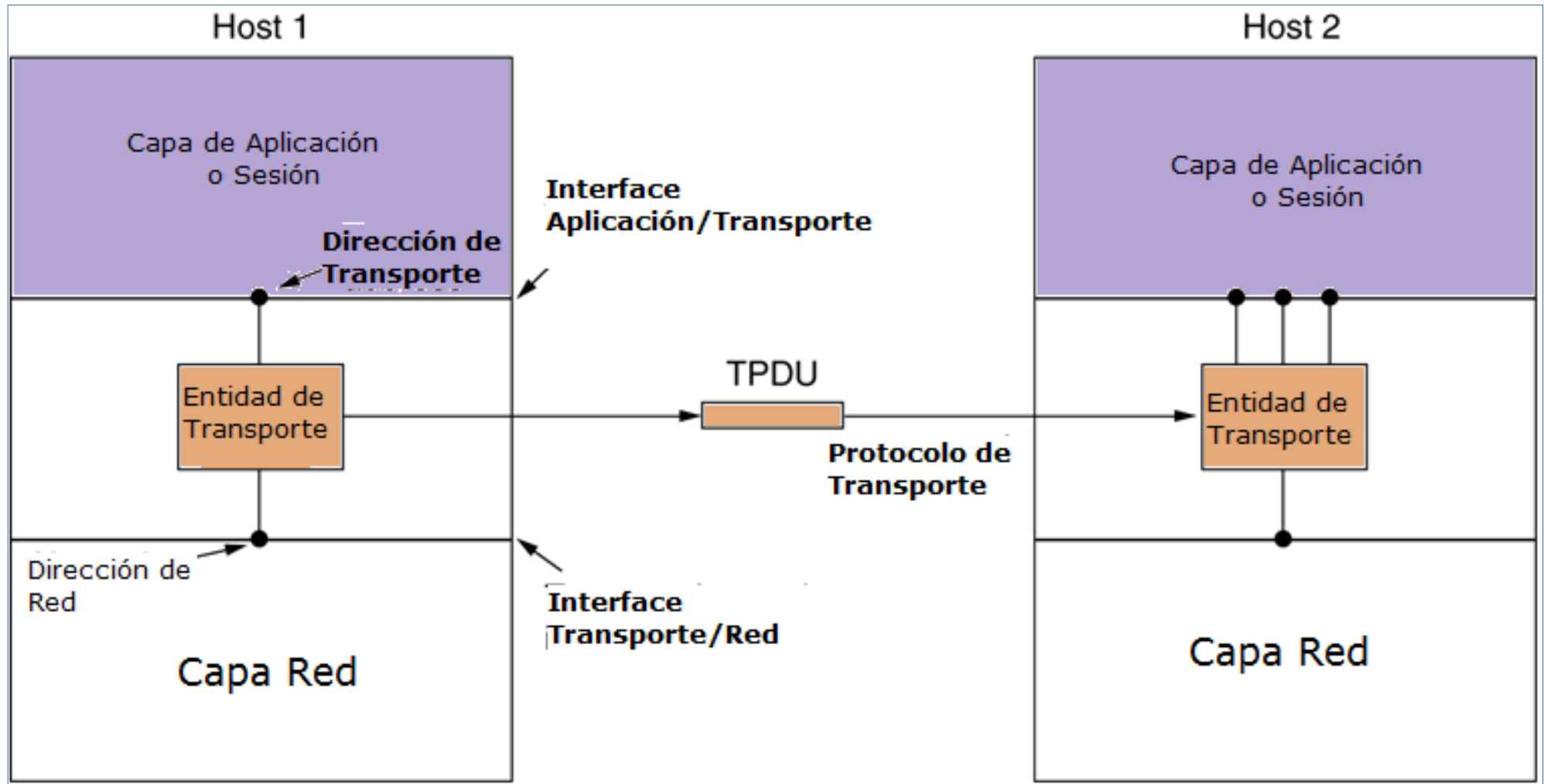


Capa de transporte proporciona la comunicación 'lógica' por encima de la física entre procesos de aplicación.

Capa Transporte – Interconexión de redes LAN y WAN. El modelo de Internet.

- Brindar servicio a los procesos de la capa de Aplicación.
- Encapsula datos en TPDU's sobre servicios de la capa de red.
- La “entidad de transporte”, se encuentra en el núcleo del sistema operativo, en procesos independientes, en un paquete de biblioteca o embebida en el software de la tarjeta de red.
- Mientras que un protocolo de la capa de transporte proporciona una comunicación lógica entre ‘procesos que se ejecutan en hosts diferentes’, un protocolo de la capa de red proporciona una comunicación lógica entre hosts.
- Los servicios de transporte pueden ser Con conexión o Sin Conexión.

Capa Transporte – Interconexión de redes LAN y WAN. El modelo de Internet.



Capa Transporte – Interconexión de redes LAN y WAN. El modelo de Internet.

- Gestiona llamadas ('Comandos') al Servicio de Transporte.
- Cada llamada corresponde exactamente a un procedimiento de biblioteca que ejecuta la primitiva.

Acciones	TPDU generada	Significado
LISTEN (Escuchar)	(ninguna)	Se bloquea hasta que algún proceso intenta el contacto.
CONNECT (Conectar)	SOLICITUD DE CONEXIÓN	Intenta activamente establecer una conexión.
SEND (Enviar)	DATOS	Envía información.
RECEIVE (Recibir)	(ninguna)	Se bloquea hasta que llega una TPDU de DATOS
DISCONNECT (Desconectar)	SOLICITUD DESCONEXIÓN	Este lado quiere liberar la conexión.

Capa Transporte – Interconexión de redes LAN y WAN. El modelo de Internet.

► Establecen mecanismos para:

- ESTABLECER CONEXIONES ENTRE NODOS
- LIBERAR SUS CONEXIONES
- MULTIPLEXION
- CONTROL DE FLUJO
- RECUPERACIÓN ANTE CAÍDAS

► El **nivel de transporte** es el cuarto nivel del modelo OSI encargado de la transferencia libre de errores de los datos entre el emisor y el receptor, aunque no estén directamente conectados, así como de mantener el flujo de la red.

- Proporciona un control de alto nivel para la transferencia de datos
- Es capaz de detectar y eliminar paquetes duplicados
- Velar por el sincronismo en la información y coordinar el reenvío de un paquete si este no ha llegado correctamente a su destino.
- Puede asignar un número único de secuencia al paquete que va a ser transmitido, para que este sea revisado en el destino por el otro nivel de transporte.

PROTOCOLLO UDP

Capa Transporte - Protocolo UDP

- Servicio de transporte sin conexión, previsto en la RFC 768.
- No emplea ninguna sincronización entre el origen y el destino.
- Trabaja con paquetes o datagramas enteros, no con bytes individuales como TCP. Una aplicación que emplea el protocolo UDP intercambia información en forma de bloques de bytes, de forma que por cada bloque de bytes enviado de la capa de aplicación a la capa de transporte, se envía un paquete UDP.
- No es fiable. No emplea control del flujo ni ordena los paquetes.
- Su gran ventaja es que provoca poca carga adicional en la red ya que es sencillo y emplea cabeceras muy simples.
- Un paquete UDP puede ser fragmentado por el protocolo IP para ser enviado fragmentado en varios paquetes IP si resulta necesario.
- Puesto que no hay conexión, un paquete UDP admite utilizar como dirección IP de destino la dirección de broadcast o de multicast de IP. Esto permite enviar un mismo paquete a varios destinos.
- DNS, SNMP, FTP, DHCP, RIP

Cabecera UDP:

+	Bits 0 - 15	16 - 31
0	Puerto origen	Puerto destino
32	Longitud del Mensaje	Suma de verificación
64	Datos	

Capa Transporte - Protocolo UDP, usos

DNSQUERY.CAP - Ethereal

File Edit View Go Capture Analyze Statistics Help

Filter: Expression... Clear Apply

No. ↓	Time	Source	Destination	Protocol	Info
1	0.000000	3com_3b:85:c1	Broadcast	ARP	who has 131.107.2.216? Tell 131.107.2.214
2	0.000000	3com_8d:95:f8	3com_3b:85:c1	ARP	131.107.2.216 is at 02:60:8c:8d:95:f8
3	0.001000	131.107.2.214	131.107.2.216	DNS	Standard query A backup.Test.com
4	0.046000	131.107.2.216	131.107.2.214	DNS	standard query response A 131.107.2.216
5	0.108000	131.107.2.214	131.107.2.216	ICMP	Echo (ping) request
6	0.109000	131.107.2.216	131.107.2.214	ICMP	Echo (ping) reply
7	1.122000	131.107.2.214	131.107.2.216	ICMP	Echo (ping) request
8	1.123000	131.107.2.216	131.107.2.214	ICMP	Echo (ping) reply
9	2.144000	131.107.2.214	131.107.2.216	ICMP	Echo (ping) request
10	2.144000	131.107.2.216	131.107.2.214	ICMP	Echo (ping) reply
11	3.165000	131.107.2.214	131.107.2.216	ICMP	Echo (ping) request
12	3.166000	131.107.2.216	131.107.2.214	ICMP	Echo (ping) reply

Frame 3 (75 bytes on wire, 75 bytes captured)

Ethernet II, Src: 3com_3b:85:c1 (02:60:8c:3b:85:c1), Dst: 3com_8d:95:f8 (02:60:8c:8d:95:f8)

Internet Protocol, Src: 131.107.2.214 (131.107.2.214), Dst: 131.107.2.216 (131.107.2.216)

User Datagram Protocol, Src Port: 1043 (1043), Dst Port: domain (53)

Source port: 1043 (1043)

Destination port: domain (53)

Length: 41

Checksum: 0x6de4 [correct]

Domain Name System (query)

Capa Transporte - Protocolo UDP, usos

PASSTHRU.CAP - Ethereal

File Edit View Go Capture Analyze Statistics Help

Filter: Expression... Clear Apply

No. ↓	Time	Source	Destination	Protocol	Info
1	0.000000	131.107.2.217	131.107.2.255	BROWSE	Get Backup List Request
2	0.001000	131.107.2.200	131.107.2.217	BROWSE	Get Backup List Response
3	0.020000	131.107.2.217	131.107.2.100	NBNS	Name query NB BACKUP<20>
4	0.024000	131.107.2.100	131.107.2.217	NBNS	Name query response NB 131.107.2.216
5	0.025000	131.107.2.217	131.107.2.216	TCP	1026 > netbios-ssn [SYN] Seq=0 Len=0 MSS=1460
6	0.026000	131.107.2.216	131.107.2.217	TCP	netbios-ssn > 1026 [SYN, ACK] Seq=0 Ack=1 win=8760
7	0.027000	131.107.2.217	131.107.2.216	TCP	1026 > netbios-ssn [ACK] Seq=1 Ack=1 win=8760 Len=

+ Frame 1 (219 bytes on wire, 219 bytes captured)

+ Ethernet II, Src: 3com_43:a1:51 (02:60:8c:43:a1:51), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

+ Internet Protocol, Src: 131.107.2.217 (131.107.2.217), Dst: 131.107.2.255 (131.107.2.255)

+ User Datagram Protocol, Src Port: netbios-dgm (138), Dst Port: netbios-dgm (138)

+ NetBIOS Datagram Service

+ SMB (Server Message Block Protocol)

+ SMB Mailslot Protocol

+ Microsoft Windows Browser Protocol

Capa Transporte – multiplexado/demultiplexado

DEMULTIPLEXACIÓN, ¿cómo trabaja ?

- El host recibe datagramas IP
- Cada datagrama tiene una dirección IP de origen, una dirección IP de destino
- Cada datagrama lleva un segmento de capa de transporte
- Cada segmento tiene origen, número de puerto de destino
- El host utiliza direcciones IP y números de puerto para asegurar la entrega al socket adecuado

Demux Sin Conexión:

El Socket UDP es identificado por: (*dirección IP destino, número de puerto destino*)

La creación de sockets:

```
DatagramSocket mySocket1 = new DatagramSocket(12534);  
DatagramSocket mySocket2 = new DatagramSocket(12535);
```

- Cuando un host recibe un segmento UDP comprueba el número de puerto de destino en el segmento
- Dirige el segmento UDP al socket con ese número de puerto
- Los Datagramas IP con diferentes direcciones IP origen y/o números de puerto de origen son dirigidos al mismo socket.

Capa Transporte –multiplexado/demultiplexado

Demultiplexado orientado a Conexión.

El Socket TCP se identifica por 4-tupla:

- **dirección IP de origen**
- **número de puerto origen**
- **dirección IP destino**
- **número de puerto destino**

▪El host que recibe el segmento utiliza los cuatro valores para direccionar la carga al zócalo correspondiente.

▪El Host servidor puede soportar muchos sockets TCP simultáneos:
cada socket es identificado por su propia 4-tupla

▪Los servidores Web tienen sockets diferentes para cada cliente que se conecta.

▪Los servidores HTTP no persistente tendrá diferentes sockets para cada solicitud

Capa Transporte – multiplexado/demultiplexado

Servicios de la capa de Transporte



PROTOCOLLO TCP

Capa Transporte – TCP Definiciones

PROTOCOLOS DE TRANSPORTE ORIENTADOS A LA CONEXIÓN

Para establecer una conexión, se envía un “Connect Request” al destino; que en caso de aceptar devuelve un “Ack”

Problema: ¿si no hay respuesta ?

- ◆ La solicitud no llegó a destino
- ◆ La respuesta no llegó a destino, con lo que volvemos a enviar la solicitud, pudiendo ser detectado.

Consecuencia: la red posee capacidad de almacenamiento.

- ◆ Se requieren buffers.

Capa Transporte – TCP Definiciones

SERVICIO DE RED CON SECUENCIAMIENTO CONFIABLE

Protocolos de transporte simples, punto a punto entre dos nodos de la misma red

Funciones:

- Direcccionamiento
- Multiplexado,
- Control de flujo,
- Inicio y terminación de conexiones,
- Recuperación ante caídas.

Capa Transporte – TCP Definiciones

► DIRECCIONAMIENTO - IDENTIFICACION

► El componente “destino” debe ser identificado de alguna forma:

- User Identification (Host, Port)
 - *Socket en TCP*
 - *Se consulta a un servidor de Nombres en caso de que se tenga configurado*
 - Identificación del tipo de transporte
 - *Especificando el protocolo (TCP, UDP)*
 - *Nombres comunes <ftp://ftp.dominio.com> ; <http://www.dominio.com> ; smtp.dominio.com*
 - Dirección de Host de un dispositivo conectado a la red
 - *Internet, Intranet*
 - Número de red
- ### ► La capa de transporte pasa el elemento a la capa de red

Capa Transporte – TCP Definiciones

CONTROL DE FLUJO

Casos:

- ▶ Tiempos de espera demasiado largos entre nodos en transmisión, retrasa la comunicación de información de control de flujo.
- ▶ Tiempos de espera variables dificultan el uso de procedimientos de timeout.
- ▶ El ancho de banda de la red no soporta lo que deseamos pasar. Se deben configurar buffers en cada lado de la conexión.
- ▶ **Porque es necesario el control de flujo:**
 - ▶ El usuario no puede seguir recibiendo.
 - ▶ La entidad de transporte no puede continuar recibiendo
 - ▶ Puede dar lugar a desbordamientos, en casos de redes “confiables” el transmisor deberá guardar TPDU's hasta tanto sean ACK's.
 - ▶ El transmisor y receptor negocian de antemano el número de TPU's a transmitir en secuencia.

Capa Transporte – TCP Definiciones

Modelo de Servicios TCP, definidos por número de puerto

Protocolos de Servicios Bien Conocidos

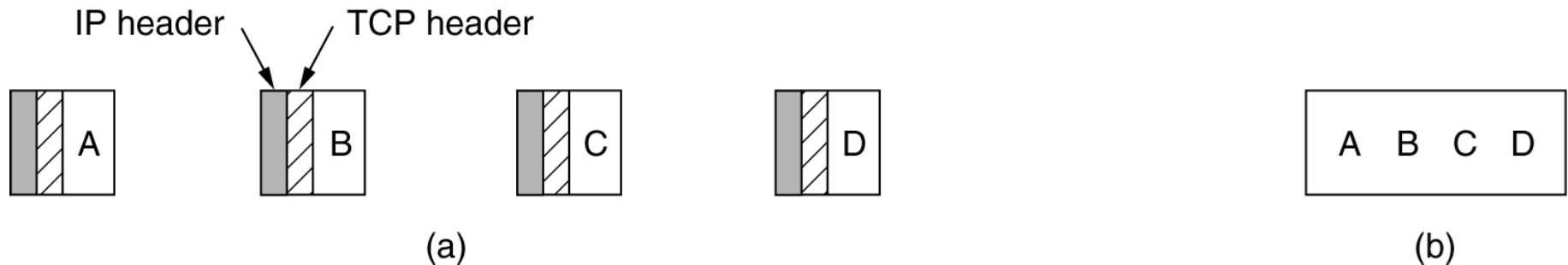
Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial File Transfer Protocol
79	Finger	Lookup info about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access

Capa Transporte - Protocolo TCP

Modelo de Servicios TCP, definidos por número de puerto

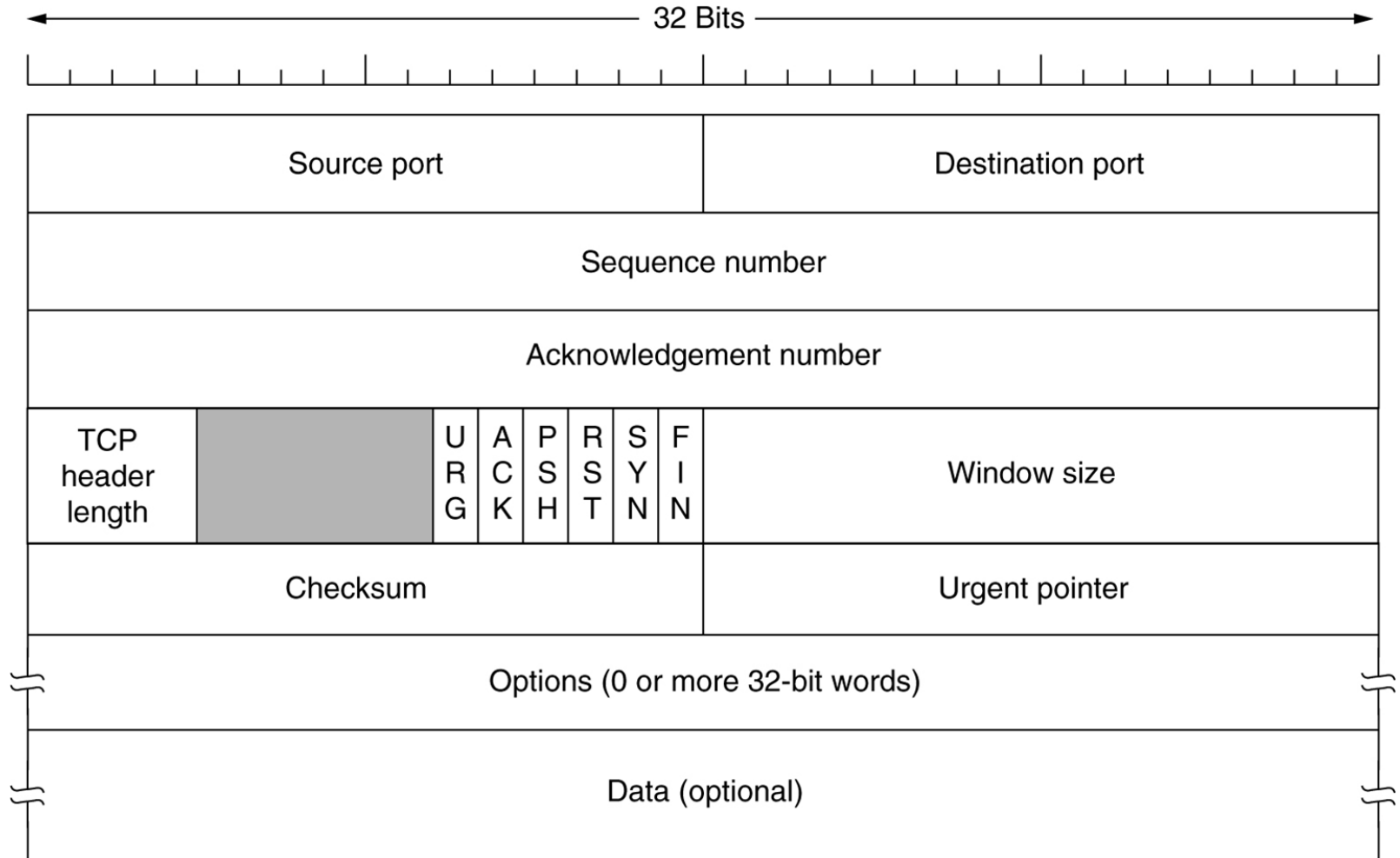
(a) Cuatro segmentos de 512 bytes son enviados en datagramas IP separados.

(b) una porción de 2048 bytes de datos se envían a la aplicación en un "READ CALL"



Capa Transporte - Protocolo TCP

Cabecera TCP



Capa Transporte – comparación de protocolos

	UDP	TCP
Tamaño header	8 Bytes	20 Bytes
Nombre de la unidad de datos	Datagrama	Segmento
Enumeración Puertos	Si	Si
Detección Errores	Opcional	Si
Fiabilidad: recuperación de errores automático	No	Si
Circuitos virtuales: Enumeración secuencial y reorganización.	No	Si
Control de Flujo	No	Si
Detección de Congestión: Ventana variable, time outs	No	Si
Flujo Múltiple	No	No

Capa Transporte - y la seguridad ?

- TCP es un protocolo 'no seguro'.
- Se hace necesario modificar el procedimiento original, agregando una capa que criptografía la sesión para hacerla segura.
- Secure Sockets Layer (SSL; en español «capa de conexión segura»)
- Transport Layer Security (TLS; en español «seguridad de la capa de transporte»)
- Aportan autenticación y privacidad de la información entre extremos sobre Internet mediante el uso de criptografía, a través de fases basado en un esquema cliente-servidor.
- Las implementaciones actuales proporcionan las siguientes opciones:
 - Criptografía de clave pública: RSA, Diffie-Hellman, DSA (Digital Signature Algorithm) o Fortezza.
 - Cifrado simétrico: RC2, RC4, IDEA (International Data Encryption Algorithm), DES (Data Encryption Standard), Triple DES y AES (Advanced Encryption Standard).
 - Funciones hash: MD5 o de la familia SHA.

Capa Transporte - Sockets, zócalos y puertos

Las API permiten desarrollar aplicaciones utilizando los servicios de TCP.

Un socket es un mecanismo de comunicación, siendo el API 4.2BSD de Unix el primero, permitiendo comunicación entre dos dominios: Internet y Unix, se ha transformado en un estándar (interfaz del zócalo).

Se definen diferentes tipo de socket en función de los servicios proporcionados:

- **Stream sockets:** orientados a conexión. Utilizan los servicios de TCP.
- **Datagram Sockets:** no orientados a conexión. Utilizan los servicios de UDP.
- **Raw Sockets:** acceso a niveles más bajos del protocolo TCP/IP.

Un zócalo (socket) es un tipo especial de archivo que un proceso usa para solicitar servicios de red al sistema operativo.

{protocolo, dirección local, proceso local }

Ejemplo TCP/IP: {tcp, 193.44.234.3, 12345}

Una conversación es el enlace de comunicaciones entre dos procesos.

Una asociación es la relación que especifica completamente los dos procesos comprendidos en una conexión:

{protocolo, dirección local, proceso - local, dirección exterior, proceso exterior}

Ejemplo TCP/IP: {tcp, 193.44.234.3, 1500, 193.44.234.5, 21}

Capa Transporte - Zócalos y Puertos

▶ Zócalos y Puertos:

▶ *Inicializar un zócalo:*

- ▶ **int sockfd** = **socket**(int family, int type, int protocol)

▶ *Registrar un zócalo en una dirección de puerto:*

- ▶ **int bind**(int sockfd, struct sockaddr *localaddr, int addrlen)

▶ *Indica disponibilidad para recibir conexiones:*

- ▶ **int listen**(int sockfd, int queue-size)

▶ *Acepta una conexión:*

- ▶ **int accept**(int sockfd, struct sockaddr *foreign-address, int addrlen)

▶ *Solicita la conexión con el servidor:*

- ▶ **int connect**(int sockfd, struct sockaddr *foreign-address, int addrlen)

▶ *Enviar o recibir datos:*

- ▶ **int read**(int sockfd, struct msghdr *msg, int len);

- ▶ **int write**(int sockfd, struct msghdr *msg, int len);

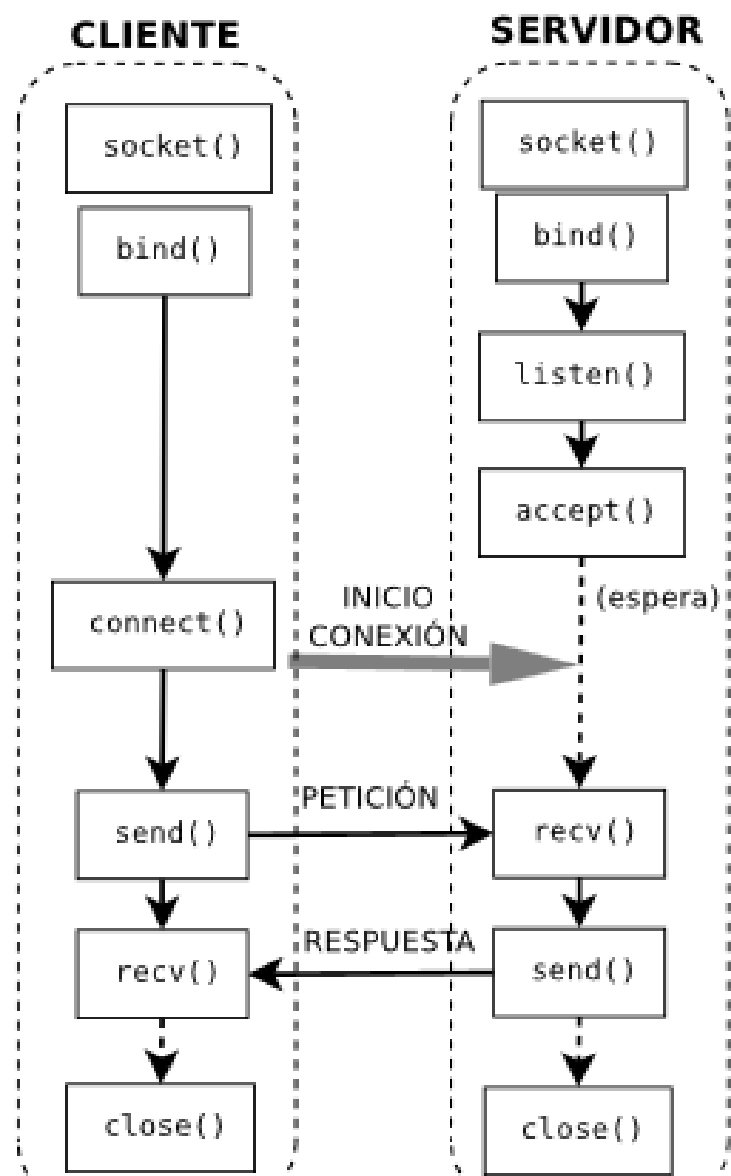
- ▶ **int send**(int sockfd, struct msghdr *msg, int len, int flags);

- ▶ **int recv**(int sockfd, struct msghdr *msg, int len, int flags);

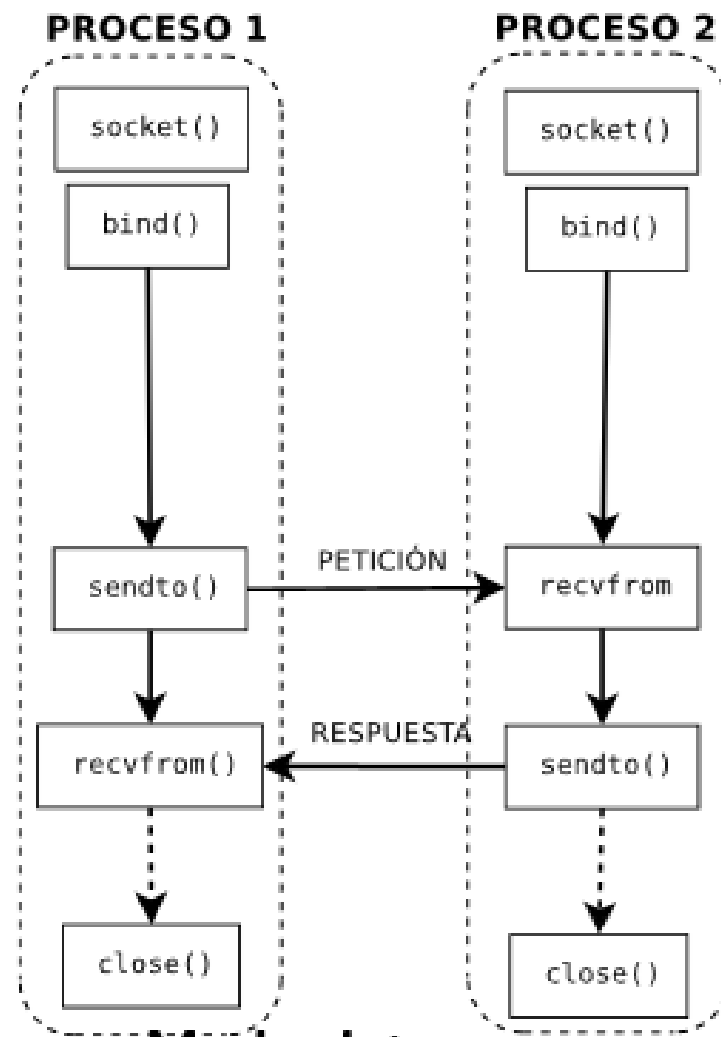
▶ *Cierra un zócalo:*

- ▶ **int close**(int sockfd)

Capa Transporte - Llamadas API, esquema de funcionamiento



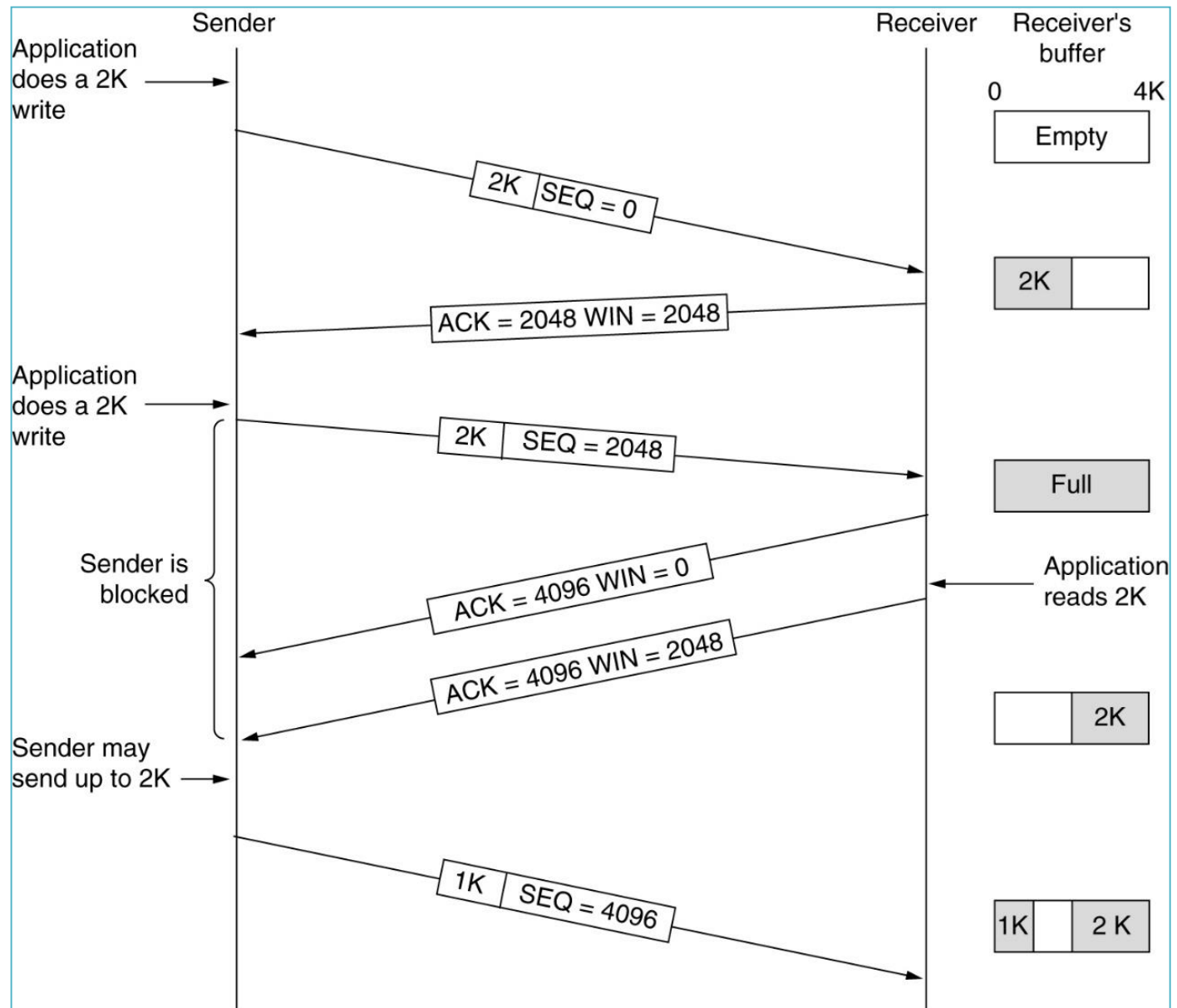
Modo stream



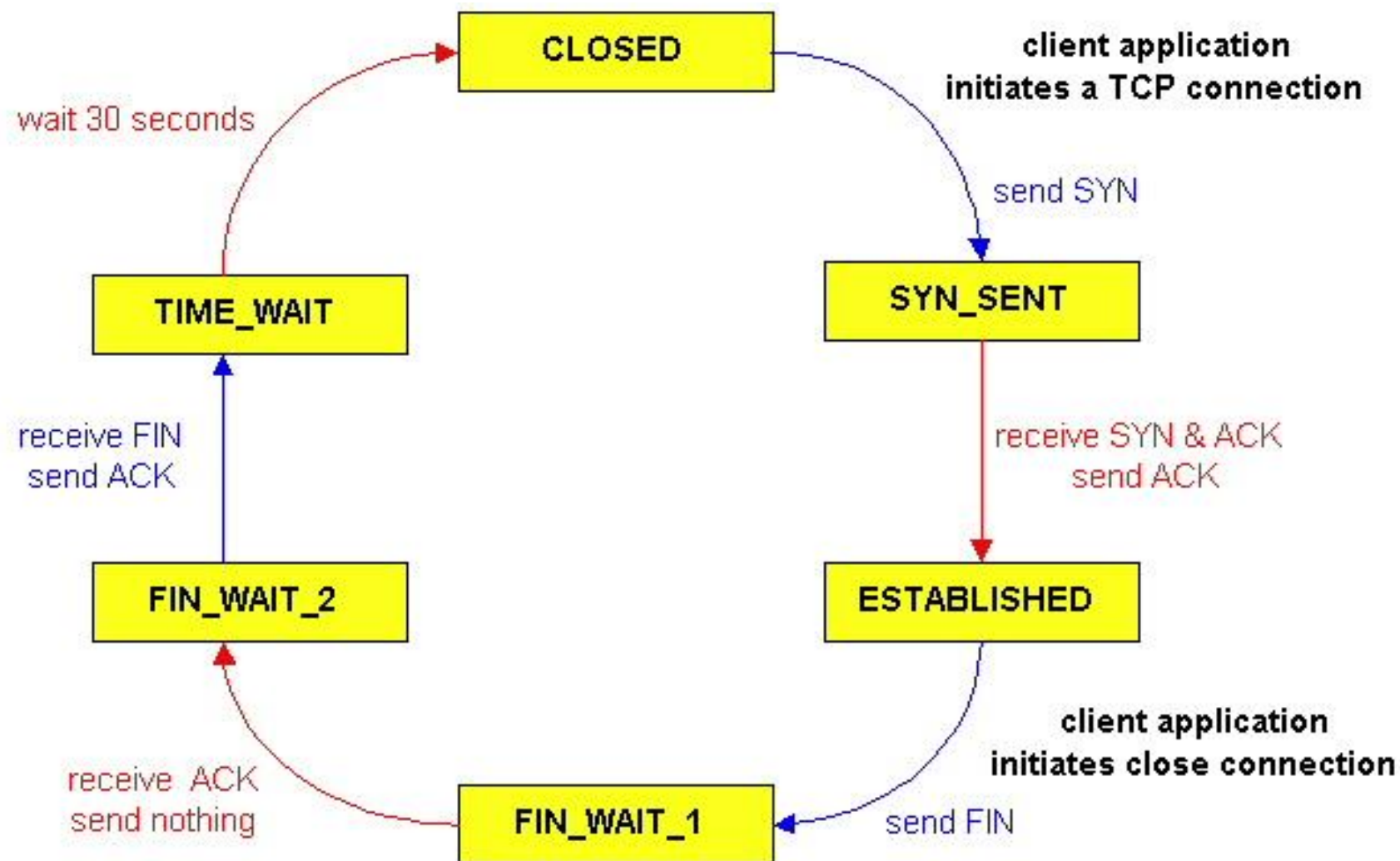
Modo datagrama

Capa Transporte - Protocolo TCP, transmisión

Gestión de Políticas de Transmisión en TCP (administración de ventanas)

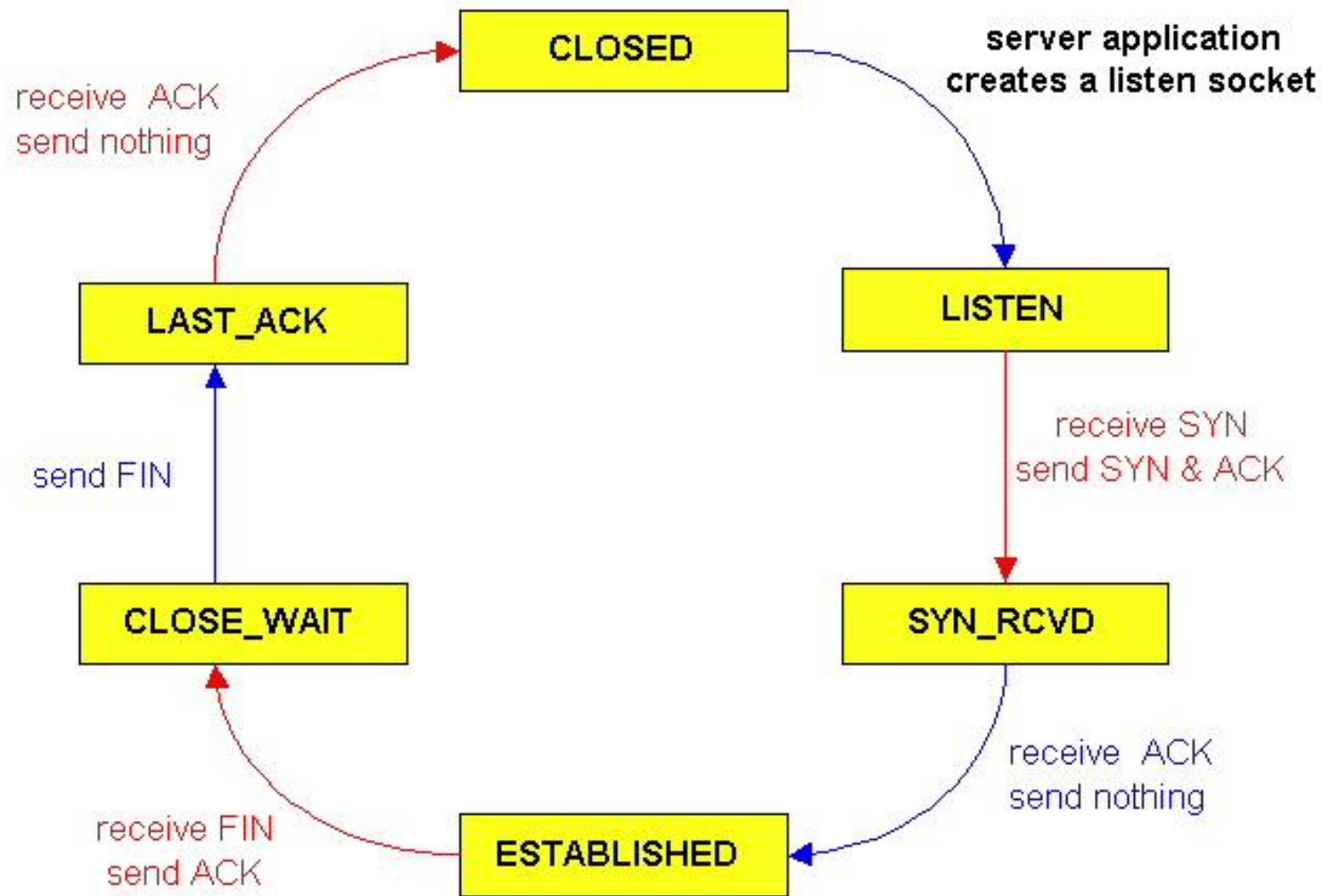


Capa Transporte - Protocolo TCP, transmisión



Ciclo de vida del TCP Cliente

Capa Transporte - Protocolo TCP, transmisión



Ciclo de vida del TCP servidor

Capa Transporte - paso de mensajes, sincronización

Ejemplo de paso de mensajes – Cliente.c

```
#include <stdio.h>
#include <Socket_Cliente.h>
#include <Socket.h>
```

```
main ()
{
/* Descriptor del socket y buffer para datos */
int Socket_Con_Servidor;
char Cadena[100];
/*
* Se abre la conexion con el servidor, pasando el nombre del ordenador
* y el servicio solicitado.
* "localhost" corresponde al nombre del mismo ordenador en el que
* estamos corriendo. Debe estar dado de alta en /etc/hosts
* "cpp_java" es un servicio dado de alta en /etc/services
*/
Socket_Con_Servidor = Abre_Conexion_Inet ("localhost", "cpp_java");
if (Socket_Con_Servidor == 1)
{
printf ("No puedo establecer conexion con el servidor\n");
exit (-1);
}
```

```
#ifndef _SOCKET_CLIENTE_H
#define _SOCKET_CLIENTE_H

int Abre_Conexion_Unix (char *Servicio);
int Abre_Conexion_Inet (char *Host_Servidor, char *Servicio);

#endif
```

```
#ifndef _SOCKET_H
#define _SOCKET_H

int Lee_Socket (int fd, char *Datos, int Longitud);
int Escribe_Socket (int fd, char *Datos, int Longitud);

#endif
```

Capa Transporte - paso de mensajes, sincronización

Continúa...

```
/*
* Se prepara una cadena con 5 caracteres y se envia, 4 letras mas
* el \0 que indica fin de cadena en C
strcpy (Cadena, "Hola");*/
fgets(Cadena, 100, stdin);
Escribe_Socket (Socket_Con_Servidor, Cadena, 100);
/*
* Se lee la informacion enviada por el servidor, que se supone es
* una cadena de 6 caracteres.
*/
Lee_Socket (Socket_Con_Servidor, Cadena, 50);
/* Se escribe en pantalla la informacion recibida del servidor */
printf ("Soy cliente, He recibido : %s\n", Cadena);
/* Se cierra el socket con el servidor */
close (Socket_Con_Servidor);
}
```


Capa Transporte - paso de mensajes, sincronización

Continúa... Servidor.c

```
#include <Socket_Servidor.h>
#include <Socket.h>
#include <string.h>
#include <stdio.h>

main ()
{
/* Descriptores de socket servidor y de socket con el cliente */
    int Socket_Servidor;
    int Socket_Cliente;
    char Cadena[100];
/* Se abre el socket servidor, con "cpp_java" dado de alta en /etc/services. */
    Socket_Servidor = Abre_Socket_Inet ("cpp_java");
    if (Socket_Servidor == -1)
    {
printf ("No se puede abrir socket servidor\n");
exit (-1);
    }
/* Se espera un cliente que quiera conectarse */
    Socket_Cliente = Acepta_Conexion_Cliente (Socket_Servidor);
    if (Socket_Servidor == -1)
    {
printf ("No se puede abrir socket de cliente\n");
exit (-1);
    }
}
```

```
#ifndef _SOCKET_SERVIDOR_H
#define _SOCKET_SERVIDOR_H

int Abre_Socket_Inet (char *Servicio);
int Abre_Socket_Unix (char *Servicio);
int Acepta_Conexion_Cliente (int Descriptor);

#endif
```

```
#ifndef _SOCKET_H
#define _SOCKET_H

int Lee_Socket (int fd, char *Datos, int Longitud);
int Escribe_Socket (int fd, char *Datos, int Longitud);

#endif
```

Capa Transporte - paso de mensajes, sincronización

Continúa... Servidor.c

```
/* Se lee la informacion del cliente, suponiendo que va a enviar 5 caracteres.*/  
Lee_Socket (Socket_Cliente, Cadena, 100);  
/* Se escribe en pantalla la informacion que se ha recibido del cliente */  
printf ("Soy Servidor, he recibido : %s\n", Cadena);  
/* Se prepara una cadena de texto para enviar al cliente. La longitud  
* de la cadena es 5 letras + \0 al final de la cadena = 6 caracteres  
strcpy (Cadena, "Cambio y fuera!");*/  
printf ("Ingrese mensaje para cliente: " );  
fgets(Cadena, 100, stdin);  
Escribe_Socket (Socket_Cliente, Cadena, 50);  
  
/* Se cierran los sockets */  
close (Socket_Cliente);  
close (Socket_Servidor);  
}
```

Compilamos todo con 'make'

En un Shell ejecutamos ./Servidor

En otro Shell ejecutamos ./Cliente

FUENTES

- Redes de Computadoras – Un enfoque descendente , Capítulo 3 y Capítulo Programación de sockets con TCP y con UDP.
- <http://ccia.ei.uvigo.es/docencia/SCS/1011/transparencias/Tema2-1.pdf>
- http://www.chuidiang.com/clinux/sockets/sockets_simp.php
- http://es.wikipedia.org/wiki/User_Datagram_Protocol